

Programación de Comunicaciones en Red

1.- Pasos básicos en la programación de sockets en el cliente

1. Creación del socket.

Se debe conocer la dirección IP del proceso servidor y el puerto por el que esté escuchando. Con este método el cliente obtiene el socket con el que podrá comunicarse con el servidor.

```
Socket cli;  
cli = new Socket("direccionIP", numeroPuerto);
```

2. Creación de los streams de entrada y de salida sobre el socket.

Se puede utilizar la clase *ObjectInputStream* para crear un stream de entrada que esté dispuesto a recibir todas las respuestas que el servidor le envíe. Esta clase permite la lectura de objetos a través del socket.

<http://docs.oracle.com/javase/1.4.2/docs/api/java/io/ObjectInputStream.html>

Con el método *getInputStream()* de la clase *Socket* se obtiene el stream de entrada para ese socket.

```
ObjectInputStream entrada = new ObjectInputStream(cli.getInputStream());
```

Para la creación de un stream de salida para enviar información al socket del servidor se puede utilizar la clase *ObjectOutputStream*.

<http://docs.oracle.com/javase/1.4.2/docs/api/java/io/ObjectOutputStream.html>

Con el método *getOutputStream()* de la clase *Socket* se obtiene el stream de salida para ese socket.

```
ObjectOutputStream salida = new ObjectOutputStream(cli.getOutputStream());
```

3. Leer y escribir a través del socket.

Para la clase `ObjectInputStream` el método utilizado es `readObject()`.

Para la clase `ObjectOutputStream` el método utilizado es `writeObject()`.

4. Cerrar los streams y el socket.

```
entrada.close();
```

```
salida.close();
```

```
cli.close();
```

Pasos básicos en la programación de sockets en el servidor

1. **Apertura del socket servidor.** Hay que indicar el número de puerto donde el servidor está escuchando nuevas peticiones de conexión por parte de los clientes.

```
ServerSocket ss = new ServerSocket(numeroPuerto);
```

2. **Esperar solicitud de conexión.**

El método `accept()` bloquea al proceso servidor esperando por una conexión por parte del proceso cliente. Lo normal es que se cree un thread para atender esa conexión, mientras el servidor puede continuar escuchando nuevas peticiones de clientes a través del `ServerSocket` (lo veremos más adelante).

```
Socket clienteSocket = ss.accept();
```

3. **Apertura del socket para la conexión.**

Cuando llega una petición de conexión, se crea un nuevo socket dentro del proceso servidor (misma línea de código del paso anterior).

4. **Creación de los streams de entrada y de salida sobre el socket.**
5. **Leer y escribir a través del socket.**
6. **Cerrar los streams y los sockets.**